

CLIENT BRIEF DOCUMENT

Project 1: Internal Knowledge Base Chatbot

Enterprise RAG Assistant - Client Kickoff Brief

Project Type	AI / GenAI Engineering - RAG Application
Primary Goal	Build a free/open-source MVP for document-based question answering
Audience	Engineer assigned to requirement analysis and solution proposal
Expected First Response	Requirement understanding document and architecture proposal within 1-2 days

Important instruction: Do not start implementation immediately. First understand the business problem, clarify scope, identify assumptions, and propose a technical solution.

1. Client Context

We are a mid-sized organization with many internal documents spread across different repositories and folders. These documents include HR policies, onboarding guides, product FAQs, technical manuals, customer support SOPs, process documents, and internal knowledge articles.

Employees currently spend a lot of time manually searching through PDFs, text files, shared folders, and old documentation. New joiners repeatedly ask senior employees the same questions, and teams sometimes rely on outdated or incomplete information.

2. Business Problem

- Knowledge is scattered across multiple documents and locations.
- Employees do not always know which document contains the correct answer.
- Manual search is slow and inefficient.
- Senior employees are repeatedly interrupted for common questions.
- New joiners take longer to become productive.
- There is no single conversational interface for approved company knowledge.
- The organization wants answers grounded only in approved documents, not generic internet knowledge.

3. Business Goal

Build an AI-powered internal knowledge assistant that allows users to ask natural language questions and receive accurate answers based only on uploaded company documents.

The assistant should reduce manual searching, improve access to internal knowledge, help new employees, and reduce repeated dependency on senior staff.

4. Example User Questions

- What is the company leave policy?
- How many annual leaves are allowed?
- What documents are required for employee onboarding?
- What is the refund process?
- How do I reset my account password?
- What are the escalation steps for customer complaints?
- What is the support SLA for priority-one issues?

5. Expected Solution

We need a web/API-based chatbot system where an admin can upload documents and users can ask questions against those documents. The system should process documents, create searchable representations, retrieve relevant content, and generate answers grounded in retrieved context.

- Accept document uploads.
- Extract and clean text from documents.
- Split documents into manageable chunks.
- Generate embeddings for chunks.
- Store embeddings in a vector database.
- Retrieve relevant chunks when a user asks a question.
- Generate an answer using only retrieved context.
- Return source references with the answer.

- Refuse to answer when the answer is not available in the uploaded documents.
- Expose APIs for upload, document listing, search, chat, and health check.

6. Key Constraint: Free / Open-Source MVP

The MVP should use free and open-source tools as much as possible. Paid APIs should not be required for the first version. If a paid API such as OpenAI or Azure OpenAI is proposed, it must be optional and clearly separated from the free/local implementation.

Preferred Technology Direction

Area	Preferred Option	Reason
Backend	Python + FastAPI	Simple API development, strong AI ecosystem
Vector DB	Qdrant / Chroma / FAISS	Open-source vector search
Embeddings	Sentence Transformers	Local embedding models
LLM	Ollama local model	Avoid paid API dependency for MVP
Metadata DB	SQLite / PostgreSQL	Store document metadata and status
Containerization	Docker + Docker Compose	Easy local setup and reproducibility
Frontend	Optional React/Next.js	API/Swagger is acceptable for MVP

7. Functional Requirements

7.1 Document Upload

- Admin should be able to upload documents.
- MVP should support PDF, TXT, and Markdown.
- Optional future support: DOCX, HTML, CSV.
- System should store document metadata: document_id, file_name, file_type, upload_time, number_of_chunks, and processing_status.

7.2 Document Processing

- Extract text from uploaded documents.
- Clean unnecessary whitespace and formatting noise.
- Split content into chunks using configurable chunk size and overlap.
- Keep metadata for each chunk: document name, document ID, chunk ID, and page number where possible.

7.3 Embedding Generation and Vector Storage

- Generate embeddings for each chunk using an open-source embedding model.
- Store embeddings and metadata in a vector database.
- Avoid duplicate processing where possible.
- Allow deletion of a document and its associated vectors.

7.4 Chat / Question Answering

- User should ask a natural language question.
- System should embed the question and retrieve top relevant chunks.
- System should build a prompt using retrieved context.
- System should generate an answer using a local LLM.
- System should return the answer and source references.

7.5 Grounding and Fallback Rule

The chatbot must answer only from retrieved document context. If the answer is not found, it should not guess. It should return:

I could not find this information in the uploaded documents.

7.6 Source References

Every answer should include sources. Example response:

Answer: Employees are allowed 20 annual leaves per year.

Sources:

1. HR Policy.pdf, page 3
2. Employee Handbook.pdf, page 7

7.7 Expected API Endpoints

Method	Endpoint	Purpose
POST	/documents/upload	Upload a document
GET	/documents	List uploaded documents
DELETE	/documents/{document_id}	Delete a document and its embeddings
POST	/chat	Ask a question and receive an answer
POST	/search	Return retrieved chunks without generating an answer
GET	/health	Basic health check

8. Non-Functional Requirements

- The system should be easy to run locally.
- The system should be containerized using Docker Compose.
- The code should be modular and understandable.
- The system should handle errors gracefully.
- The chatbot should avoid hallucination by using retrieved context only.
- The API should return meaningful error messages.
- The design should allow future replacement of local LLM with OpenAI or Azure OpenAI.
- The design should allow future frontend integration.
- The design should allow future authentication and role-based access.

9. Logging and Debugging Requirements

- Log when a document is uploaded.
- Log when document processing starts and completes.
- Log number of chunks created.
- Log embedding generation status.
- Log user question received.
- Log retrieved chunk IDs and similarity scores if available.
- Log LLM response time.
- Log errors during extraction, retrieval, and generation.

10. Testing Expectations

- Document upload test.

- Text extraction test.
- Chunking test.
- Embedding and retrieval test.
- Chat endpoint test.
- Answer-not-found test.
- Document deletion test if deletion is implemented.

11. Out of Scope for MVP

- User authentication.
- Role-based access control.
- Multi-tenant support.
- Production cloud deployment.
- Advanced UI.
- Fine-tuning.
- Voice input.
- Real-time document sync.
- Complex agentic workflows.
- Payment or subscription features.

12. Expected First Deliverables After 1-2 Days

Do not start implementation immediately. First provide a project understanding document and proposed solution. The expected first deliverables are:

12.1 Requirement Understanding Document

- Explain the business problem in your own words.
- Explain what the system is supposed to achieve.
- List functional requirements.
- List non-functional requirements.
- List assumptions.
- List out-of-scope items.
- List risks and open questions.

12.2 Clarifying Questions

- Who are the users of the chatbot?
- What document formats are required?
- Should answers always include citations?
- Should the chatbot support conversation history?
- Should old documents be deleted or versioned?
- What should happen if two documents contain conflicting information?
- Should the system support multiple departments?
- What level of accuracy is expected?
- Is frontend required for the MVP?
- Should local LLM be mandatory?

12.3 Proposed MVP Scope

- Define what will be built in version 1.
- Separate must-have features from nice-to-have features.
- Mention what will not be included in the MVP.

12.4 Proposed Technical Architecture

- High-level architecture diagram.
- Main system components.
- Data flow from document upload to answer generation.
- Technology choices and justification.
- Alternatives considered.

12.5 Proposed API Design

- List API endpoints.
- Provide request and response examples.
- Define important error responses.

12.6 Development Plan

- Milestone 1: Project setup and API skeleton.
- Milestone 2: Document upload and text extraction.
- Milestone 3: Chunking and embeddings.
- Milestone 4: Vector database integration.
- Milestone 5: Chat endpoint and answer generation.
- Milestone 6: Source citations and fallback handling.
- Milestone 7: Tests, Docker, and documentation.

12.7 Risks and Mitigation

Risk	Expected Mitigation
Local LLM may produce weak answers.	Use stronger local model if available, improve prompt, or make paid API optional.
PDF extraction may fail for scanned PDFs.	Mention scanned PDFs as out of scope for MVP or propose OCR later.
Large documents may slow processing.	Add file size limits and background processing later.
Retrieval quality may be poor initially.	Tune chunk size, overlap, embedding model, top_k, and metadata filtering.
Embeddings may miss exact keyword matches.	Consider hybrid search as a future enhancement.
Hallucination may occur.	Use strict prompt, retrieved-context-only rule, and answer-not-found fallback.
Docker setup may be difficult.	Provide clear Docker Compose and README instructions.

13. Final Project Success Criteria

- A user can upload documents.
- The system can process and chunk documents.
- The system can store embeddings.
- The system can retrieve relevant chunks for a question.
- The chatbot answers using only document context.
- The chatbot returns sources.
- The chatbot refuses to answer when context is missing.
- The app can run locally through Docker Compose.
- The README is clear and complete.
- The code is modular and understandable.
- The project can be explained confidently in an interview.

14. Instruction to the Assigned Engineer

Your task is not to impress the client with code immediately. Your task is to understand the problem, ask good questions, define scope, and propose a practical solution.

For the next session, come back as an engineer assigned to this client project. Bring your requirement understanding document, clarifying questions, proposed MVP, architecture, API design, tech stack, development plan, and risks. Do not write production code yet. We need to review how you think before we review how you code.

Client Review Note: The proposal will be reviewed from two angles: business understanding and technical feasibility.